

```

In [ ]: # =====
# Project:
# DINOv3 with a Simple Linear Head for Skin Disease Classification
#
# Description:
# This script uses a pre-trained DINOv3 Vision Transformer as a frozen feature
# extractor. A simple, trainable linear classification head is attached to the
# [CLS] token output for the final prediction. This serves as a strong baseline
# for transfer learning. The pipeline preloads the dataset into RAM for speed.
#
# Author:
# Gemini (Adapted from DINOv3 DSAF script)
#
# Date:
# September 24, 2025
# =====

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import os
import numpy as np
import pandas as pd
from tqdm import tqdm
import math
import warnings
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

# Core ML/DL Libraries
from torchvision import transforms, datasets
from torch.utils.data import Dataset, DataLoader
from transformers import AutoModel
from torch.amp import GradScaler, autocast

# Evaluation & Utilities
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    roc_auc_score,
    cohen_kappa_score,
    matthews_corrcoef,
)
from sklearn.preprocessing import label_binarize

# Suppress non-critical warnings
warnings.filterwarnings("ignore", category=UserWarning, module="PIL")
os.environ["TOKENIZERS_PARALLELISM"] = "false"

```

```

# =====
# I. SETUP AND CONFIGURATION
# =====

config = {
    # --- Hardware & Performance Tuning ---
    "DEVICE": torch.device("cuda" if torch.cuda.is_available() else "cpu"),
    "AMP_ENABLED": True,
    "NUM_WORKERS": 0, # MUST be 0 for in-memory dataset on Windows

    # --- File Paths ---
    "DATA_DIR": r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset",
    "RESULTS_DIR": "full_Dataset_DINOv3_Simple", # <-- MODIFIED

    # --- Model & Training Hyperparameters ---
    "MODEL_NAME": "facebook/dinov3-vitb16-pretrain-lvd1689m",
    "IMAGE_SIZE": 224,
    "BATCH_SIZE": 128,
    "NUM_EPOCHS": 200,
    "LEARNING_RATE": 1e-4,
    "WEIGHT_DECAY": 1e-4,
    "WARMUP_EPOCHS": 10,
    "EARLY_STOPPING_PATIENCE": 20,
    "LABEL_SMOOTHING": 0.1,

    # --- Reproducibility ---
    "RANDOM_SEED": 42
}

# --- Create results directory ---
os.makedirs(config["RESULTS_DIR"], exist_ok=True)
config["MODEL_SAVE_PATH_BEST"] = os.path.join(config["RESULTS_DIR"], "DINOv3_Simple")

# --- Set Seed for Reproducible Experiments ---
def set_seed(seed):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = True
    torch.backends.cudnn.deterministic = False

set_seed(config["RANDOM_SEED"])

# --- Initial Setup Verification ---
print("\n" + "="*80)
print(" " * 15 + "DINOv3 Simple Head Classification Pipeline Initialization") # <--
print("="*80)
print(f"\n[INFO] PyTorch Version: {torch.__version__}")
print(f"[INFO] Data Loading Mode: In-Memory (Preloading to RAM)")
print(f"[INFO] Using Device: {config['DEVICE']}")
print(f"[INFO] Early Stopping Patience: {config['EARLY_STOPPING_PATIENCE']} epochs")
print(f"[INFO] Training for up to {config['NUM_EPOCHS']} epochs.")
print("="*80)

```

```

# =====
# II. DATA PREPARATION & HANDLING
# =====
class InMemoryDataset(Dataset):
    def __init__(self, data_list, transform=None):
        self.data_list = data_list
        self.transform = transform

    def __len__(self):
        return len(self.data_list)

    def __getitem__(self, idx):
        image, label = self.data_list[idx]
        if self.transform:
            image = self.transform(image)
        return image, label

cpu_transforms = transforms.Compose([
    transforms.Resize((config["IMAGE_SIZE"], config["IMAGE_SIZE"])),
    transforms.ToTensor()
])

class GPUAugmentation(nn.Module):
    def __init__(self):
        super().__init__()
        self.transforms = nn.Sequential(
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomRotation(degrees=15),
            transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.2, hue=0.1),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        )

    @torch.no_grad()
    def forward(self, x):
        return self.transforms(x)

print("\n" + "="*80)
print(" " * 28 + "Phase 1: Data Preparation")
print("="*80)

try:
    temp_datasets = {
        x: datasets.ImageFolder(os.path.join(config["DATA_DIR"], x))
        for x in ['train', 'val', 'test']
    }
    class_names = temp_datasets['train'].classes
    num_classes = len(class_names)
    config["NUM_CLASSES"] = num_classes

    print(f"[INFO] Discovered {num_classes} classes: {' '.join(class_names)}")
    print("[INFO] Pre-loading all images into RAM. This may take a moment...")

    preloaded_data = {'train': [], 'val': [], 'test': []}
    for split in ['train', 'val', 'test']:
        for path, label in tqdm(temp_datasets[split].samples, desc=f"--> Loading {split}"):
            with Image.open(path).convert("RGB") as img:

```

```

        image_tensor = cpu_transforms(img)
        preloaded_data[split].append((image_tensor, label))

image_datasets = {
    'train': InMemoryDataset(preloaded_data['train']),
    'val': InMemoryDataset(preloaded_data['val']),
    'test': InMemoryDataset(preloaded_data['test'])
}

dataloaders = {
    x: DataLoader(
        image_datasets[x],
        batch_size=config["BATCH_SIZE"],
        shuffle=(x == 'train'),
        num_workers=config["NUM_WORKERS"],
        pin_memory=True
    )
    for x in ['train', 'val', 'test']
}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}

print(f"[SUCCESS] All image data has been pre-loaded into memory.")
print(f" - Dataset sizes: Train={dataset_sizes['train']}, Val={dataset_sizes['val']}, Test={dataset_sizes['test']}")

except FileNotFoundError as e:
    print(f"\n[ERROR] Dataset not found. Please check the path in config['DATA_DIR']")
    print(f"Details: {e}")
    exit()

# =====
# III. MODEL, OPTIMIZER, AND SCHEDULER INITIALIZATION
# =====

# <-- START: MODIFIED MODEL DEFINITION WITH SIMPLE HEAD -->
class SimpleHead(nn.Module):
    """A simple linear classification head."""
    def __init__(self, hidden_size=768, num_classes=5, dropout=0.1):
        super().__init__()
        self.classifier = nn.Sequential(
            nn.LayerNorm(hidden_size),
            nn.Linear(hidden_size, hidden_size // 2),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_size // 2, num_classes)
        )

    def forward(self, hidden_states):
        # Use only the [CLS] token for classification
        cls_token = hidden_states[:, 0]
        logits = self.classifier(cls_token)
        return logits

class DINOv3ForSkinClassification(nn.Module):
    def __init__(self, model_name, num_classes):
        super().__init__()
        self.dinov3 = AutoModel.from_pretrained(model_name)

```

```

    # Freeze the backbone
    for param in self.dinov3.parameters():
        param.requires_grad = False

    hidden_size = self.dinov3.config.hidden_size
    # Attach the simple trainable head
    self.classifier = SimpleHead(hidden_size=hidden_size, num_classes=num_class)

    def forward(self, pixel_values):
        self.dinov3.eval()
        with torch.no_grad():
            outputs = self.dinov3(pixel_values=pixel_values)
            last_hidden_state = outputs.last_hidden_state
            logits = self.classifier(last_hidden_state)
        return logits

model = DINOv3ForSkinClassification(model_name=config["MODEL_NAME"], num_classes=co
# <-- END: MODIFIED MODEL DEFINITION -->

criterion = nn.CrossEntropyLoss(label_smoothing=config["LABEL_SMOOTHING"])
optimizer = optim.AdamW(model.classifier.parameters(), lr=config["LEARNING_RATE"],

lr_lambda = lambda epoch: (epoch + 1) / config["WARMUP_EPOCHS"] if epoch < config["
    0.5 * (1 + math.cos(math.pi * (epoch - config["WARMUP_EPOCHS"])) / (config["NUM_
scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

scaler = GradScaler(enabled=config["AMP_ENABLED"])
gpu_augmenter = GPUAugmentation().to(config["DEVICE"])
gpu_normalizer = gpu_augmenter.transforms[-1]

# =====
# IV. TRAINING & VALIDATION LOOPS
# =====
def train_one_epoch():
    model.train()
    running_loss = 0.0
    correct_preds = 0
    progress_bar = tqdm(dataloaders['train'], desc="--> Training", leave=False)

    for inputs, labels in progress_bar:
        inputs = inputs.to(config["DEVICE"], non_blocking=True)
        labels = labels.to(config["DEVICE"], non_blocking=True)

        inputs = gpu_augmenter(inputs)

        optimizer.zero_grad(set_to_none=True)

        with autocast(device_type=config["DEVICE"].type, dtype=torch.float16, enabl
            outputs = model(pixel_values=inputs)
            loss = criterion(outputs, labels)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

```

```

_, preds = torch.max(outputs, 1)
running_loss += loss.item() * inputs.size(0)
correct_preds += torch.sum(preds == labels.data)
progress_bar.set_postfix(loss=f"{loss.item():.4f}")

epoch_loss = running_loss / dataset_sizes['train']
epoch_acc = correct_preds.double() / dataset_sizes['train']
return epoch_loss, epoch_acc.item()

def validate_one_epoch():
    model.eval()
    running_loss = 0.0
    correct_preds = 0
    progress_bar = tqdm(dataloaders['val'], desc="--> Validating", leave=False)

    with torch.no_grad():
        for inputs, labels in progress_bar:
            inputs = inputs.to(config["DEVICE"])
            labels = labels.to(config["DEVICE"])

            inputs = gpu_normalizer(inputs)

            with autocast(device_type=config["DEVICE"].type, dtype=torch.float16, e
                outputs = model(pixel_values=inputs)
                loss = criterion(outputs, labels)

            _, preds = torch.max(outputs, 1)
            running_loss += loss.item() * inputs.size(0)
            correct_preds += torch.sum(preds == labels.data)

    epoch_loss = running_loss / dataset_sizes['val']
    epoch_acc = correct_preds.double() / dataset_sizes['val']
    return epoch_loss, epoch_acc.item()

# =====
# V. EVALUATION & VISUALIZATION FUNCTIONS
# =====
def plot_training_history(history):
    save_path = os.path.join(config["RESULTS_DIR"], "training_history.png")
    epochs = range(1, len(history['train_loss']) + 1)
    plt.style.use('seaborn-v0_8-darkgrid')
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 7))
    ax1.plot(epochs, history['train_loss'], 'o-', label='Train Loss')
    ax1.plot(epochs, history['val_loss'], 'o-', label='Validation Loss')
    ax1.set_title('Model Loss Over Epochs')
    ax1.set_xlabel('Epochs')
    ax1.set_ylabel('Loss')
    ax1.legend()
    ax2.plot(epochs, [acc * 100 for acc in history['train_acc']], 'o-', label='Trai
    ax2.plot(epochs, [acc * 100 for acc in history['val_acc']], 'o-', label='Valida
    ax2.set_title('Model Accuracy Over Epochs')
    ax2.set_xlabel('Epochs')
    ax2.set_ylabel('Accuracy (%)')
    ax2.legend()

```

```

fig.suptitle('DINOv3 Simple Head Training & Validation History') # <-- MODIFIED
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(save_path, dpi=300)
plt.show()
plt.close()
print(f"\nSaved training history plot to: {save_path}")

def plot_scalar_metrics(metrics):
    save_path = os.path.join(config["RESULTS_DIR"], "scalar_metrics.png")
    plt.style.use('seaborn-v0_8-whitegrid')
    fig, ax = plt.subplots(figsize=(10, 6))
    metrics_df = pd.DataFrame(list(metrics.items()), columns=['Metric', 'Value'])
    sns.barplot(x='Value', y='Metric', data=metrics_df, ax=ax, palette='viridis', h
    ax.set_title('Overall Scalar Performance Metrics')
    ax.set_xlabel('Value')
    ax.set_ylabel('')
    ax.set_xlim(0, 1)
    for p in ax.patches:
        width = p.get_width()
        ax.text(width + 0.01, p.get_y() + p.get_height() / 2, f'{width:.4f}', va='c
    plt.tight_layout()
    plt.savefig(save_path, dpi=300)
    plt.show()
    plt.close()
    print(f"\nSaved scalar metrics plot to: {save_path}")

def perform_full_evaluation(model_path, loader, class_names):
    print("\n" + "="*80)
    print("OVERALL TEST PERFORMANCE")
    print("="*80)
    try:
        eval_model = DINOv3ForSkinClassification(config["MODEL_NAME"], num_classes=
        eval_model.load_state_dict(torch.load(model_path, map_location=config["DEVI
        eval_model.to(config["DEVICE"]))
        eval_model.eval()
    except FileNotFoundError:
        print(f"\n[ERROR] Model file not found at {model_path}. Evaluation skipped.
        return

    all_labels, all_preds, all_probs = [], [], []
    with torch.no_grad():
        for inputs, labels in tqdm(loader, desc="--> Evaluating on Test Set"):
            inputs = inputs.to(config["DEVICE"])
            inputs = gpu_normalizer(inputs)
            outputs = eval_model(pixel_values=inputs)
            probs = F.softmax(outputs, dim=1)
            _, predicted = torch.max(outputs.data, 1)
            all_labels.extend(labels.cpu().numpy())
            all_preds.extend(predicted.cpu().numpy())
            all_probs.extend(probs.cpu().numpy())

    all_labels, all_preds, all_probs = np.array(all_labels), np.array(all_preds), n
    print(classification_report(all_labels, all_preds, target_names=class_names, di

    labels_bin = label_binarize(all_labels, classes=range(len(class_names)))
    macro_roc_auc = roc_auc_score(labels_bin, all_probs, average='macro', multi_cla

```

```

scalar_metrics = {
    "Accuracy": (all_labels == all_preds).mean(),
    "Macro ROC AUC": macro_roc_auc,
    "Cohen's Kappa": cohen_kappa_score(all_labels, all_preds),
    "Matthews CorrCoef": matthews_corrcoef(all_labels, all_preds),
}

print("\n--- Overall Scalar Metrics ---")
for name, value in scalar_metrics.items(): print(f"{name}: {value:.4f}")

cm_path = os.path.join(config["RESULTS_DIR"], "confusion_matrix.png")
plt.style.use('default')
plt.figure(figsize=(10, 8))
sns.heatmap(confusion_matrix(all_labels, all_preds, normalize='true'), annot=True)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Normalized Confusion Matrix')
plt.savefig(cm_path, dpi=300)
plt.show()
plt.close()
print(f"\nSaved normalized confusion matrix to: {cm_path}")

roc_path = os.path.join(config["RESULTS_DIR"], "roc_curves.png")
plt.style.use('seaborn-v0_8-darkgrid')
plt.figure(figsize=(12, 9))
colors = matplotlib.colormaps.get('viridis', len(class_names))
for i, (class_name, color) in enumerate(zip(class_names, colors.colors)):
    fpr, tpr, _ = roc_curve(labels_bin[:, i], all_probs[:, i])
    plt.plot(fpr, tpr, color=color, lw=2.5, label=f'ROC for {class_name} (AUC =
plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Chance')
plt.xlim([-0.02, 1.0]); plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve - One-vs-Rest'); plt.legend(loc="lower right")
plt.savefig(roc_path, dpi=300); plt.show(); plt.close()
print(f"\nSaved ROC curves plot to: {roc_path}")

plot_scalar_metrics(scalar_metrics)
print("\nDone.")

# =====
# VI. MAIN EXECUTION BLOCK
# =====
if __name__ == '__main__':
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_loss = float('inf')
    epochs_no_improve = 0
    best_epoch = 0

    print("\n" + "="*80)
    print(" " * 29 + "Phase 2: Model Training")
    print("="*80)
    print(f"[INFO] Starting training for up to {config['NUM_EPOCHS']} epochs...")
    print(f"[INFO] Early stopping is active with patience={config['EARLY_STOPPING_P

```

```

for epoch in range(config["NUM_EPOCHS"]):
    train_loss, train_acc = train_one_epoch()
    val_loss, val_acc = validate_one_epoch()
    scheduler.step()

    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

    current_lr = optimizer.param_groups[0]['lr']

    print(
        f"Epoch [{epoch+1:03d}/{config['NUM_EPOCHS']:03d}] | "
        f"LR: {current_lr:.2e} | "
        f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
        f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}"
    )

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        epochs_no_improve = 0
        best_epoch = epoch + 1
        torch.save(model.state_dict(), config["MODEL_SAVE_PATH_BEST"])
        print(f"    > Validation loss improved to {best_val_loss:.4f}. Saving b
    else:
        epochs_no_improve += 1
        print(f"    > Val loss did not improve for {epochs_no_improve} epoch(s)

    if epochs_no_improve >= config['EARLY_STOPPING_PATIENCE']:
        print(f"\n[INFO] Early stopping triggered after {epoch + 1} epochs.")
        print(f"    Best model was saved at epoch {best_epoch} with validati
        break

print(f"\n[SUCCESS] Model training completed.")
plot_training_history(history)

perform_full_evaluation(
    model_path=config["MODEL_SAVE_PATH_BEST"],
    loader=dataloaders['test'],
    class_names=class_names
)

```



```
Epoch [007/200] | LR: 8.00e-05 | Train Loss: 0.8102 | Train Acc: 0.8510 | Val Loss: 0.8243 | Val Acc: 0.8400
  > Validation loss improved to 0.8243. Saving best model.

Epoch [008/200] | LR: 9.00e-05 | Train Loss: 0.7346 | Train Acc: 0.8800 | Val Loss: 0.7657 | Val Acc: 0.8600
  > Validation loss improved to 0.7657. Saving best model.

Epoch [009/200] | LR: 1.00e-04 | Train Loss: 0.6818 | Train Acc: 0.8890 | Val Loss: 0.7301 | Val Acc: 0.8680
  > Validation loss improved to 0.7301. Saving best model.

Epoch [010/200] | LR: 1.00e-04 | Train Loss: 0.6572 | Train Acc: 0.8945 | Val Loss: 0.7022 | Val Acc: 0.8840
  > Validation loss improved to 0.7022. Saving best model.

Epoch [011/200] | LR: 1.00e-04 | Train Loss: 0.6329 | Train Acc: 0.9045 | Val Loss: 0.6898 | Val Acc: 0.8680
  > Validation loss improved to 0.6898. Saving best model.

Epoch [012/200] | LR: 1.00e-04 | Train Loss: 0.6182 | Train Acc: 0.9095 | Val Loss: 0.6800 | Val Acc: 0.8920
  > Validation loss improved to 0.6800. Saving best model.

Epoch [013/200] | LR: 9.99e-05 | Train Loss: 0.6094 | Train Acc: 0.9195 | Val Loss: 0.6753 | Val Acc: 0.9000
  > Validation loss improved to 0.6753. Saving best model.

Epoch [014/200] | LR: 9.99e-05 | Train Loss: 0.6045 | Train Acc: 0.9150 | Val Loss: 0.6709 | Val Acc: 0.9080
  > Validation loss improved to 0.6709. Saving best model.

Epoch [015/200] | LR: 9.98e-05 | Train Loss: 0.5877 | Train Acc: 0.9280 | Val Loss: 0.6647 | Val Acc: 0.9000
  > Validation loss improved to 0.6647. Saving best model.

Epoch [016/200] | LR: 9.98e-05 | Train Loss: 0.5885 | Train Acc: 0.9330 | Val Loss: 0.6666 | Val Acc: 0.8920
  > Val loss did not improve for 1 epoch(s).

Epoch [017/200] | LR: 9.97e-05 | Train Loss: 0.5800 | Train Acc: 0.9305 | Val Loss: 0.6599 | Val Acc: 0.9000
  > Validation loss improved to 0.6599. Saving best model.

Epoch [018/200] | LR: 9.96e-05 | Train Loss: 0.5780 | Train Acc: 0.9340 | Val Loss: 0.6628 | Val Acc: 0.9040
  > Val loss did not improve for 1 epoch(s).

Epoch [019/200] | LR: 9.94e-05 | Train Loss: 0.5726 | Train Acc: 0.9300 | Val Loss: 0.6552 | Val Acc: 0.9000
  > Validation loss improved to 0.6552. Saving best model.
```

```
Epoch [020/200] | LR: 9.93e-05 | Train Loss: 0.5643 | Train Acc: 0.9420 | Val Loss: 0.6606 | Val Acc: 0.8880
> Val loss did not improve for 1 epoch(s).

Epoch [021/200] | LR: 9.92e-05 | Train Loss: 0.5584 | Train Acc: 0.9405 | Val Loss: 0.6563 | Val Acc: 0.8920
> Val loss did not improve for 2 epoch(s).

Epoch [022/200] | LR: 9.90e-05 | Train Loss: 0.5581 | Train Acc: 0.9430 | Val Loss: 0.6549 | Val Acc: 0.9000
> Validation loss improved to 0.6549. Saving best model.

Epoch [023/200] | LR: 9.88e-05 | Train Loss: 0.5513 | Train Acc: 0.9415 | Val Loss: 0.6583 | Val Acc: 0.8920
> Val loss did not improve for 1 epoch(s).

Epoch [024/200] | LR: 9.87e-05 | Train Loss: 0.5475 | Train Acc: 0.9460 | Val Loss: 0.6542 | Val Acc: 0.9040
> Validation loss improved to 0.6542. Saving best model.

Epoch [025/200] | LR: 9.85e-05 | Train Loss: 0.5447 | Train Acc: 0.9455 | Val Loss: 0.6570 | Val Acc: 0.8920
> Val loss did not improve for 1 epoch(s).

Epoch [026/200] | LR: 9.83e-05 | Train Loss: 0.5434 | Train Acc: 0.9460 | Val Loss: 0.6504 | Val Acc: 0.9040
> Validation loss improved to 0.6504. Saving best model.

Epoch [027/200] | LR: 9.80e-05 | Train Loss: 0.5330 | Train Acc: 0.9520 | Val Loss: 0.6546 | Val Acc: 0.8920
> Val loss did not improve for 1 epoch(s).

Epoch [028/200] | LR: 9.78e-05 | Train Loss: 0.5354 | Train Acc: 0.9510 | Val Loss: 0.6508 | Val Acc: 0.8960
> Val loss did not improve for 2 epoch(s).

Epoch [029/200] | LR: 9.76e-05 | Train Loss: 0.5369 | Train Acc: 0.9495 | Val Loss: 0.6564 | Val Acc: 0.8880
> Val loss did not improve for 3 epoch(s).

Epoch [030/200] | LR: 9.73e-05 | Train Loss: 0.5306 | Train Acc: 0.9580 | Val Loss: 0.6505 | Val Acc: 0.8960
> Val loss did not improve for 4 epoch(s).

Epoch [031/200] | LR: 9.70e-05 | Train Loss: 0.5288 | Train Acc: 0.9565 | Val Loss: 0.6536 | Val Acc: 0.8960
> Val loss did not improve for 5 epoch(s).

Epoch [032/200] | LR: 9.67e-05 | Train Loss: 0.5254 | Train Acc: 0.9525 | Val Loss: 0.6477 | Val Acc: 0.9000
> Validation loss improved to 0.6477. Saving best model.
```

```
Epoch [033/200] | LR: 9.64e-05 | Train Loss: 0.5254 | Train Acc: 0.9545 | Val Loss: 0.6554 | Val Acc: 0.8800
> Val loss did not improve for 1 epoch(s).

Epoch [034/200] | LR: 9.61e-05 | Train Loss: 0.5225 | Train Acc: 0.9605 | Val Loss: 0.6500 | Val Acc: 0.8840
> Val loss did not improve for 2 epoch(s).

Epoch [035/200] | LR: 9.58e-05 | Train Loss: 0.5198 | Train Acc: 0.9590 | Val Loss: 0.6543 | Val Acc: 0.8840
> Val loss did not improve for 3 epoch(s).

Epoch [036/200] | LR: 9.55e-05 | Train Loss: 0.5201 | Train Acc: 0.9600 | Val Loss: 0.6595 | Val Acc: 0.8880
> Val loss did not improve for 4 epoch(s).

Epoch [037/200] | LR: 9.51e-05 | Train Loss: 0.5204 | Train Acc: 0.9640 | Val Loss: 0.6554 | Val Acc: 0.8880
> Val loss did not improve for 5 epoch(s).

Epoch [038/200] | LR: 9.47e-05 | Train Loss: 0.5120 | Train Acc: 0.9635 | Val Loss: 0.6545 | Val Acc: 0.8960
> Val loss did not improve for 6 epoch(s).

Epoch [039/200] | LR: 9.44e-05 | Train Loss: 0.5140 | Train Acc: 0.9615 | Val Loss: 0.6546 | Val Acc: 0.8920
> Val loss did not improve for 7 epoch(s).

Epoch [040/200] | LR: 9.40e-05 | Train Loss: 0.5105 | Train Acc: 0.9610 | Val Loss: 0.6467 | Val Acc: 0.9000
> Validation loss improved to 0.6467. Saving best model.

Epoch [041/200] | LR: 9.36e-05 | Train Loss: 0.5110 | Train Acc: 0.9610 | Val Loss: 0.6551 | Val Acc: 0.8920
> Val loss did not improve for 1 epoch(s).

Epoch [042/200] | LR: 9.32e-05 | Train Loss: 0.5058 | Train Acc: 0.9630 | Val Loss: 0.6471 | Val Acc: 0.8960
> Val loss did not improve for 2 epoch(s).

Epoch [043/200] | LR: 9.27e-05 | Train Loss: 0.5036 | Train Acc: 0.9690 | Val Loss: 0.6498 | Val Acc: 0.8920
> Val loss did not improve for 3 epoch(s).

Epoch [044/200] | LR: 9.23e-05 | Train Loss: 0.5051 | Train Acc: 0.9660 | Val Loss: 0.6502 | Val Acc: 0.8880
> Val loss did not improve for 4 epoch(s).

Epoch [045/200] | LR: 9.19e-05 | Train Loss: 0.5068 | Train Acc: 0.9660 | Val Loss: 0.6594 | Val Acc: 0.8920
> Val loss did not improve for 5 epoch(s).
```

```
Epoch [046/200] | LR: 9.14e-05 | Train Loss: 0.5029 | Train Acc: 0.9675 | Val Loss: 0.6524 | Val Acc: 0.8920
> Val loss did not improve for 6 epoch(s).

Epoch [047/200] | LR: 9.09e-05 | Train Loss: 0.5019 | Train Acc: 0.9710 | Val Loss: 0.6546 | Val Acc: 0.8920
> Val loss did not improve for 7 epoch(s).

Epoch [048/200] | LR: 9.05e-05 | Train Loss: 0.5039 | Train Acc: 0.9660 | Val Loss: 0.6612 | Val Acc: 0.8880
> Val loss did not improve for 8 epoch(s).

Epoch [049/200] | LR: 9.00e-05 | Train Loss: 0.4927 | Train Acc: 0.9755 | Val Loss: 0.6516 | Val Acc: 0.8960
> Val loss did not improve for 9 epoch(s).

Epoch [050/200] | LR: 8.95e-05 | Train Loss: 0.5008 | Train Acc: 0.9690 | Val Loss: 0.6647 | Val Acc: 0.8840
> Val loss did not improve for 10 epoch(s).

Epoch [051/200] | LR: 8.89e-05 | Train Loss: 0.5032 | Train Acc: 0.9720 | Val Loss: 0.6584 | Val Acc: 0.8920
> Val loss did not improve for 11 epoch(s).

Epoch [052/200] | LR: 8.84e-05 | Train Loss: 0.4944 | Train Acc: 0.9740 | Val Loss: 0.6598 | Val Acc: 0.8840
> Val loss did not improve for 12 epoch(s).

Epoch [053/200] | LR: 8.79e-05 | Train Loss: 0.4895 | Train Acc: 0.9730 | Val Loss: 0.6603 | Val Acc: 0.8920
> Val loss did not improve for 13 epoch(s).

Epoch [054/200] | LR: 8.73e-05 | Train Loss: 0.4937 | Train Acc: 0.9715 | Val Loss: 0.6522 | Val Acc: 0.8920
> Val loss did not improve for 14 epoch(s).

Epoch [055/200] | LR: 8.68e-05 | Train Loss: 0.4890 | Train Acc: 0.9720 | Val Loss: 0.6540 | Val Acc: 0.8880
> Val loss did not improve for 15 epoch(s).

Epoch [056/200] | LR: 8.62e-05 | Train Loss: 0.4959 | Train Acc: 0.9690 | Val Loss: 0.6550 | Val Acc: 0.8960
> Val loss did not improve for 16 epoch(s).

Epoch [057/200] | LR: 8.56e-05 | Train Loss: 0.4849 | Train Acc: 0.9755 | Val Loss: 0.6577 | Val Acc: 0.8840
> Val loss did not improve for 17 epoch(s).

Epoch [058/200] | LR: 8.51e-05 | Train Loss: 0.4912 | Train Acc: 0.9735 | Val Loss: 0.6651 | Val Acc: 0.8880
> Val loss did not improve for 18 epoch(s).
```

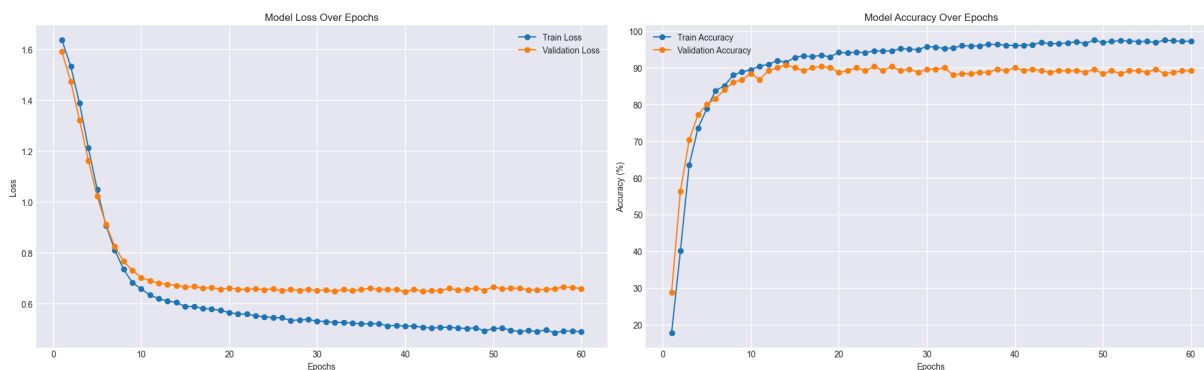
```
Epoch [059/200] | LR: 8.45e-05 | Train Loss: 0.4918 | Train Acc: 0.9720 | Val Loss: 0.6630 | Val Acc: 0.8920
> Val loss did not improve for 19 epoch(s).
```

```
Epoch [060/200] | LR: 8.39e-05 | Train Loss: 0.4885 | Train Acc: 0.9725 | Val Loss:
0.6588 | Val Acc: 0.8920
> Val loss did not improve for 20 epoch(s).
```

```
[INFO] Early stopping triggered after 60 epochs.  
Best model was saved at epoch 40 with validation loss: 0.6467
```

[SUCCESS] Model training completed.

DINOv3 Simple Head Training & Validation History



Saved training history plot to: full_results_DINOv3_SimpleHead\training_history.png

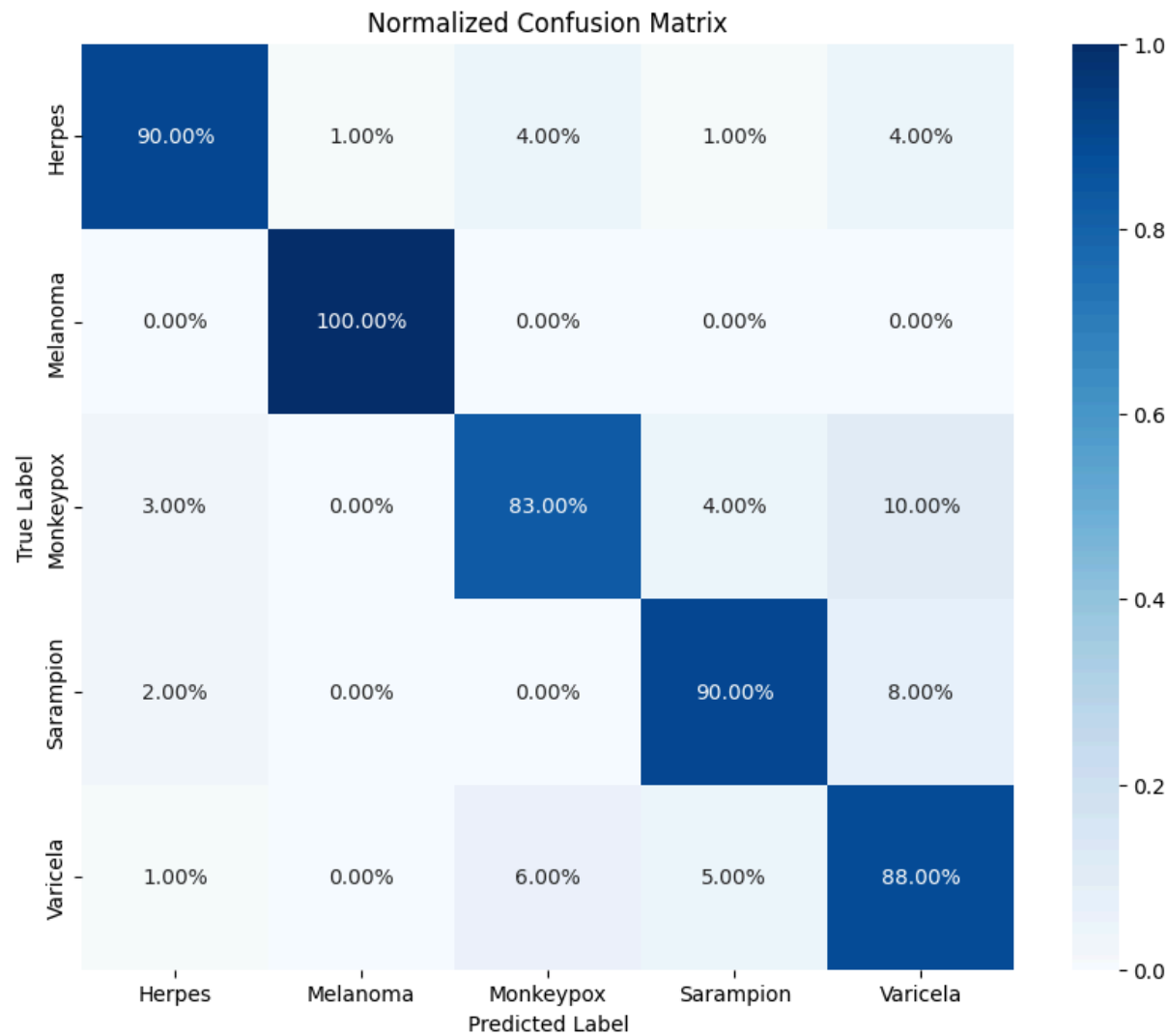
OVERALL TEST PERFORMANCE

```
--> Evaluating on Test Set: 100%|██████████| 4/4 [00:04<00:00, 1.18s/it]
```

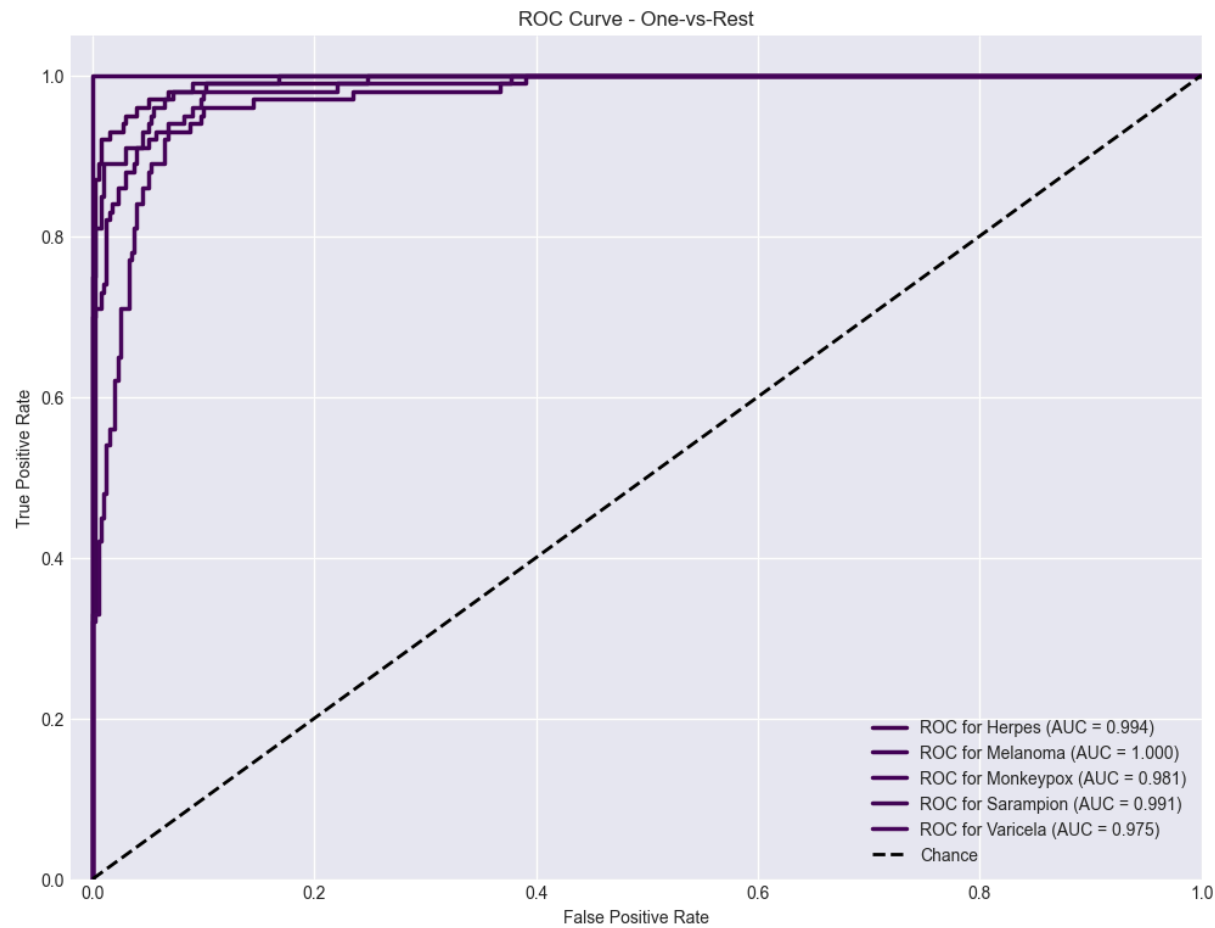
	precision	recall	f1-score	support
Herpes	0.9375	0.9000	0.9184	100
Melanoma	0.9901	1.0000	0.9950	100
Monkeypox	0.8925	0.8300	0.8601	100
Sarampion	0.9000	0.9000	0.9000	100
Varicela	0.8000	0.8800	0.8381	100
accuracy			0.9020	500
macro avg	0.9040	0.9020	0.9023	500
weighted avg	0.9040	0.9020	0.9023	500

```
--- Overall Scalar Metrics ---
```

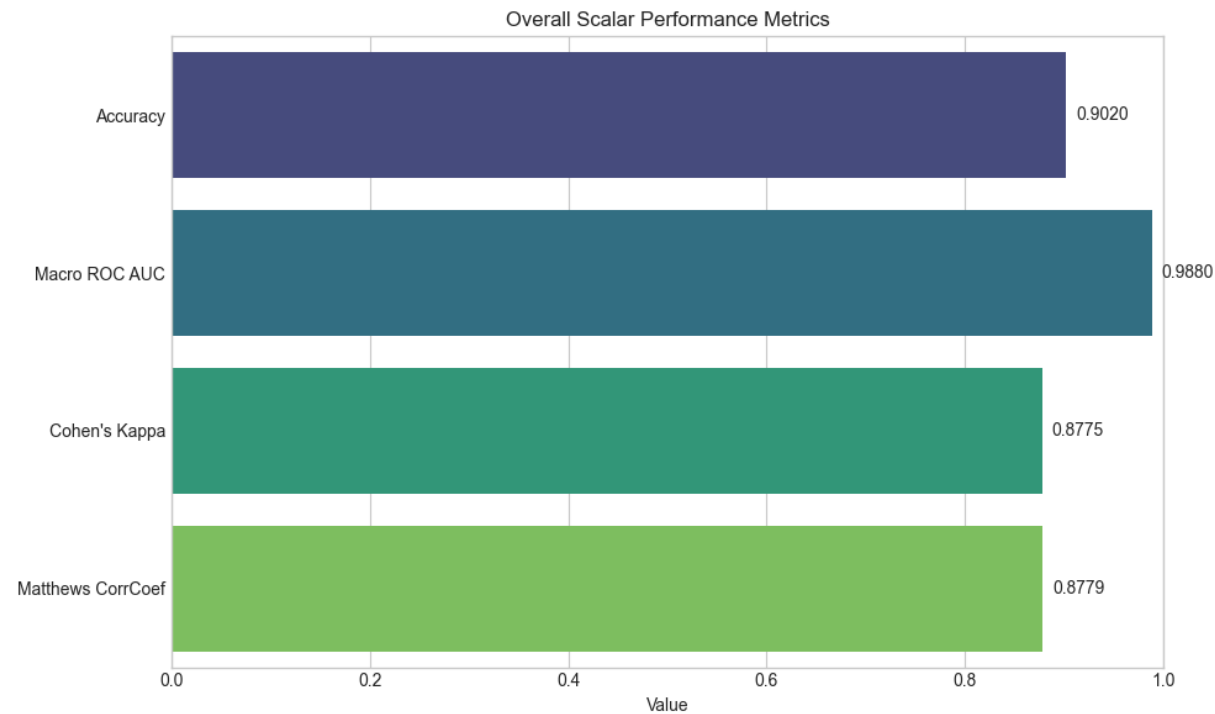
Accuracy: 0.9020
Macro ROC AUC: 0.9880
Cohen's Kappa: 0.8775
Matthews CorrCoef: 0.8779



Saved normalized confusion matrix to: full_results_DINOV3_SimpleHead\confusion_matrix.png



Saved ROC curves plot to: full_results_DINOV3_SimpleHead\roc_curves.png



Saved scalar metrics plot to: full_results_DINOV3_SimpleHead\scalar_metrics.png

Done.

```
In [ ]:
```

